

FR3 仿真控制系统 · 期末项目方案

课程: 机器人建模与控制 团队: 6 人 · 4 周 仿真平台: MuJoCo + Python 机械臂: Franka Research 3(7-DOF)

一、整体思路与答辩叙事

项目定位

从零搭建一个 FR3 仿真控制系统, 完整覆盖建模、控制、规划三大课程核心, 自主推导与实现关键算法, 并扩展到 learning-based 控制方法。

答辩一句话

"我们没有把 FR3 当黑箱去调用, 而是从 DH 参数开始, 自己推导运动学、自己写 IK 求解器、自己实现并对比了多种控制器, 最后用 learning 方法做了一个对比扩展。"

为什么选这个方案(现实考量)

为什么纯仿真? 真机环境(franka_ros2、libfranka)在 4 周内配通的风险过高。纯仿真把"工程踩坑"的时间换成"算法实现与对比"的时间, 与课程主题更匹配。

为什么不用 MoveIt 之类的现成框架? MoveIt 把 IK、规划、控制都封装成黑箱, 跟课程核心内容(建模、控制律推导)关系反而弱。自己写一遍才是这门课该做的事。

为什么选 MuJoCo?

- 装起来简单(`pip install mujoco`), 无环境踩坑
- 官方 `mujoco_menagerie` 提供高质量 FR3 模型
- 接触动力学精度高, 适合做控制对比
- Python 接口干净, 代码量小

为什么不进视觉模块? 仿真里物体位置完全已知, 视觉感知是"自找麻烦"。视觉的故事在真机上才有意义, 纯仿真下进主线性价比低。

为什么把学习方法保留为扩展? Learning 是当下机器人领域的主流方向之一, 在报告里加一个"经典控制 vs 学习控制"的对比, 既体现广度又有深度。

项目结构总览

M1 建模	DH 参数 + 正运动学 + 雅可比	课程核心
M2 逆运动学	数值解 + 解析解(可选) + 冗余处理	课程核心 · 主要深度抓手
M3 轨迹规划	关节空间 + 笛卡尔空间 + 多种插值法	课程核心
M4 控制器	PD + 计算力矩 + 阻抗, 三种对比	课程核心 · 主要深度抓手
M5 仿真环境	MuJoCo + FR3 + 桌面任务场景	工程基础
M6 任务集成	reaching + pick-and-place demo	综合演示
M7 学习扩展	Behavior Cloning vs 经典控制对比	扩展

M1-M6 是主线, 必做; M7 是扩展, 争取做。

二、各模块详细规划

M1 · 运动学建模

任务: 推导 FR3 的 DH 参数(改进 DH 推荐), 实现正运动学和雅可比矩阵。

技术栈: Python + NumPy + SymPy(符号推导, 可选)

基本目标

- ☐ FR3 改进 DH 参数表, 与官方文档/URDF 交叉验证
- ☐ 正运动学函数: 输入 7 个关节角, 输出末端位姿
- ☐ 几何雅可比矩阵 $J(q) \in \mathbb{R}^{(6 \times 7)}$ 实现

☐ **验证:**与 MuJoCo 内置的 FK 结果对比,误差应在数值精度内

报告产出

- DH 参数表 + 坐标系示意图
- FK 推导过程
- 雅可比推导过程
- 与 MuJoCo 对比的误差表

M2 • 逆运动学(主要深度抓手)

任务:实现 FR3 的逆运动学求解,处理 7-DOF 冗余。

技术栈:Python + NumPy + SciPy

基本目标

- ☐ **数值解 1:**Jacobian Pseudoinverse(伪逆迭代)
- ☐ **数值解 2:**Damped Least Squares(DLS,处理奇异点)
- ☐ 冗余自由度的零空间投影,实现"在末端不变的前提下移动肘部"
- ☐ **对比实验:**三个测试位姿下,两种数值解的迭代次数、收敛性、计算时间
- ☐ 奇异点附近的稳定性测试

加分项

- ☐ **解析解:**基于 7-DOF 冗余分解(如肘角参数化),实现一个半解析的 IK
- ☐ 在零空间里加优化目标(避关节极限、避奇异)

为什么这是深度抓手

- 7-DOF 冗余 IK 是经典难题,做透了报告含金量很高
- 多种方法对比是天然的实验设计
- 完全自己写,无外部依赖

M3 • 轨迹规划

任务:实现关节空间和笛卡尔空间的轨迹规划。

技术栈:Python + NumPy

基本目标

- ☐ **关节空间:**五次多项式插值(起止位置/速度/加速度可控)
- ☐ **关节空间:**梯形速度规划(LSPB)
- ☐ **笛卡尔空间:**位置直线插值 + 姿态 SLERP
- ☐ 两类规划的速度、加速度曲线可视化
- ☐ **对比:**同一起止点,两种关节空间方法的差异

加分项

- ☐ B 样条 / 多段轨迹拼接
- ☐ 在线避障(简单情况)

M4 • 控制器设计与对比(主要深度抓手)

任务:实现三种控制器,在同一任务上对比。

技术栈:Python + MuJoCo

基本目标

- ☐ **控制器 1:**关节空间 PD 控制(带重力补偿)
- ☐ **控制器 2:**计算力矩控制(Computed Torque,Inverse Dynamics)
- ☐ **控制器 3:**笛卡尔空间阻抗控制
- ☐ 推导每个控制器的控制律,在报告中给出
- ☐ **对比实验:**同一轨迹下三种控制器的跟踪误差、能耗、抗干扰能力
- ☐ 干扰实验:在仿真中给末端施加外力,看响应差异

报告产出

- 三种控制律的数学推导
- 跟踪误差曲线对比
- 干扰响应对比
- 参数调节经验总结

为什么这是深度抓手

- 三种控制器是课程的重点
- 自己实现 + 对比 = 标准的 "控制系统设计" 叙事
- 实验数据丰富, 报告章节饱满

M5 · 仿真环境搭建

任务: 搭建可复用的 FR3 仿真环境。

技术栈: MuJoCo + mujoco_menagerie/franka_fr3 + Python

基本目标

- ☐ MuJoCo 加载 FR3 + Franka Hand
- ☐ 桌面 + 物体场景(几个不同形状的物体)
- ☐ 统一的控制接口(扭矩/位置/速度都能切换)
- ☐ 数据记录工具(关节状态、末端位姿、控制量、时间戳)
- ☐ 可视化工具(matplotlib 画轨迹、误差曲线)

M6 · 任务集成

任务: 用前面的模块完成两个端到端任务。

基本目标

- ☐ **任务 1 · Reaching:** 从 home 出发到达指定位姿, 展示 IK + 轨迹规划 + 控制
- ☐ **任务 2 · Pick-and-Place:** 抓取桌面物体放到指定区域, 展示完整流水线
- ☐ 两个任务都用三种控制器跑一遍, 记录成功率与轨迹质量
- ☐ 录制 demo 视频

M7 · 学习方法扩展

任务: 用 Behavior Cloning 学一个策略, 与经典控制器对比。

技术栈: Python + PyTorch + MuJoCo

基本目标

- ☐ 用 M2-M4 的经典控制器作为 scripted expert, 在 M6 任务上生成 100+ 条轨迹
- ☐ 训练 BC 网络(MLP): 观测 → 动作
- ☐ 在仿真中评估 BC 策略, 与经典控制器对比
- ☐ 拟合精度分析、失败模式分析

降级方案(若 BC 训不出可用策略)

- 报告改写成 "BC 拟合精度分析 + 失败模式讨论", 仍是合格的学术内容

三、4 周时间表

第 1 周 · 建模与环境

角色	任务
A、F	M5 仿真环境搭建,MuJoCo + FR3 加载,数据记录工具
B	M1 DH 参数推导 + FK 实现
C	M1 雅可比推导与实现 + 与 B 对接验证
D	M7 准备:MuJoCo 接口熟悉 + PyTorch 环境
E	M2 数值 IK 启动:Pseudoinverse + DLS

周末检查点:MuJoCo 环境跑通;FK + 雅可比与 MuJoCo 对比误差 < 1e-6;IK 初版能跑。

第 2 周 · IK 与轨迹规划

角色	任务
A	M3 轨迹规划(关节空间 + 笛卡尔)+ 集成接口
B	M4 控制器 1(PD + 重力补偿)实现
C	M4 控制器 2(计算力矩)实现 + 推导文档
D	M7 数据收集 pipeline 搭建
E	M2 完成 + 冗余零空间处理 + 奇异点测试
F	报告骨架 + M1、M2 章节起草

周末检查点:IK 全部完成并出对比数据;PD 和计算力矩两种控制器在 reaching 上跑通;轨迹规划接口可用。

第 3 周 · 控制器对比 + 任务集成 + 学习扩展

角色	任务
A	M6 任务集成,两个任务跑通
B	M4 控制器对比实验,数据收集
C	M4 控制器 3(阻抗)实现 + 干扰实验
D	M7 BC 训练 + 评估
E	M2 加分项(解析解或零空间优化)+ 协助 M4
F	M3、M4 章节撰写 + 视频素材开始拍

周末检查点(go/no-go):

- 三种控制器对比数据齐全 → 否则砍 M7 加分项,集中收尾
- 两个任务 demo 跑通
- BC 至少跑通 pipeline

第 4 周 · 收尾

角色	任务
全员周一-周三	补漏实验,但 周三后不再做新实验
F 主导	报告整合 + PPT 主架构
D、E	各自模块的可视化与图表
A、B、C	视频剪辑、答辩排练
周末	最终排练 + 提交

四、人员分工

角色分配

代号	角色	主责模块	主要技术栈
A	队长 / 系统集成	M5 环境 + M6 集成 + 全程协调	Python、MuJoCo、项目管理
B	建模与控制 1	M1 FK + M4 PD 控制器	NumPy、SymPy、控制理论
C	建模与控制 2	M1 雅可比 + M4 计算力矩 + 阻抗	NumPy、控制理论、动力学
D	学习扩展	M7 全栈	PyTorch、MuJoCo、ML
E(你)	IK 与轨迹规划	M2 逆运动学 + M3 轨迹规划	NumPy、SciPy、数值方法
F	文档 / 机动	全程文档 + 救火 + 视频	Markdown、LaTeX、剪辑

F 不是打杂。F 从第 1 周开始介入,第 2 周末必须有报告骨架和已完成章节的初稿。**F 给学习能力强、心态稳的人。**

报告与 PPT 分工

章节	主笔	协作
摘要 / 项目概览	A	F
系统架构与仿真环境(M5)	A	—
运动学建模(M1)	B	C
雅可比推导(M1)	C	B
逆运动学(M2)	E	—
轨迹规划(M3)	A	E
控制器设计与对比(M4)	B + C	—
任务集成与实验(M6)	A	F 整理数据
学习方法扩展(M7)	D	—
结论与展望	F + A	全员 review
演示视频	F 剪辑	全员提供素材
答辩 PPT	F 排版	各章节作者提供内容

写作时间线

- 第 2 周末:F 完成报告骨架,M1、M2 章节初稿
- 第 3 周末:M3、M4 章节初稿
- 第 4 周:整合 + 实验章节 + PPT + 视频

五、需要队内敲定的讨论点

会议时按这个顺序过一遍:

1. **MuJoCo Python binding 用官方 mujoco 还是 dm_control ?** 推荐官方 mujoco ,资料新、依赖少。
2. **DH 参数用标准 DH 还是改进 DH(Modified DH)?** 推荐改进 DH——FR3 官方文档用的是改进 DH,可以直接对照验证。
3. **控制器编程接口统一吗?** 建议:所有控制器统一为 `controller(q, dq, q_des, dq_des, ...)` `→ tau` 形式,A 第 2 周中锁定接口规范。
4. **第 3 周末 go/no-go 决策由谁拍板?** 建议队长 A 拍板,F 提供进度数据。
5. **报告交付物格式与字数要求?** 待确认课程要求。
6. **如果时间不够,M7 砍还是 M4 阻抗砍?** 建议优先保 M4 阻抗(课程内容),砍 M7(扩展)。
7. **是否给最后真机预留缓冲?** 原计划"仿真效果好就上真机"——4 周内不太现实,建议**作为答辩后/未来工作来讲**,不进入 4 周计划。

六、风险清单

风险	概率	对策
第 1 周 MuJoCo 模型加载有问题	低	A 第 1 周专注此事,1 天搞不定就求助
IK 数值解收敛慢/不稳	中	E 备好两种方法(伪逆 + DLS),交叉验证
计算力矩控制需要的动力学参数从哪来?	中	直接读 MuJoCo 的 mj_rne 或 mj_fullM,不自己推
BC 训不出可用策略	高	已预设降级方案
报告写晚	高	F 第 2 周必须开始
模块之间接口不统一	中	A 第 2 周中锁定

七、执行纪律

1. M1-M6 是主线,M7 是扩展,任何时候主线优先
2. 控制器接口第 2 周中锁定,后续不许改
3. 报告第 2 周开始写
4. 第 3 周末 go/no-go 由队长拍板
5. 第 4 周周三后不再做新实验
6. 加分项不做不丢分,基本目标必须达成
7. 任何环节延期超过 1 天必须报警